MDPs and POMDPs

Lecture 23

CS 4880 Kirk Duran

Lecture 22 Gave Us a Way to Compare Consequences

We combined:

- beliefs about an uncertain world;
- utilities that express preferences over outcomes;
- expected utility to compare a route choice;
- value of information to decide whether a reading was worth buying.

Those examples ended after one route segment.

Key idea

Today we model the next state, the decisions available there, and the consequences of continuing to act.

Today: Two Models for Sequential Decisions

Part 1: MDPs

We observe the current modeled state. Actions have uncertain outcomes. Plan for accumulated reward and choose an action for each state.

Part 2: POMDPs

The current physical state may be hidden. Maintain a belief, plan for future observations, and choose an action for each belief.

Key idea

Both models optimize a strategy over time. Partial observability changes what the agent knows when choosing.

The Problem: A Locally Good Action Can Create a Bad Future

Shortcut now

Immediate reward: $+6$.
Fast if the plaza stays clear, but entering it may trap the robot behind dense pedestrian traffic.

Detour now

Immediate reward: $+2$.
Slower right now, but it moves the robot into a predictable corridor with many good future choices.

Key idea

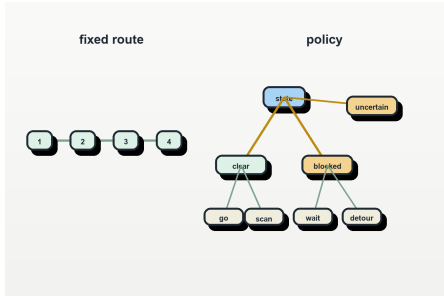
The quality of an action depends on both its immediate outcome and the future states it makes likely.

A Fixed Route Cannot Specify Every Future Response

A path commits to one sequence of moves. But after each action:

- pedestrians may block the intended route;
- wheel slip may put the robot elsewhere;
- a door may open or close.

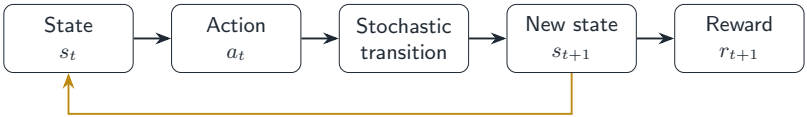
The robot must choose again from the situation it actually reaches.



Key idea

A policy specifies how to respond to the states reached, including outcomes that differ from the expected route.

The MDP Loop



Symbols: t: current time step; s_t: current state; a_t: chosen action; s_{t+1}: resulting next state; r_{t+1}: reward received for that transition.

Key idea

A Markov Decision Process formalizes repeated decision making when actions have uncertain consequences and future rewards matter.

A Markov Decision Process

$$\boxed{MDP = (S, A, T, R, \gamma)}$$

S	states: what situation is the robot in?
A	actions: what can the robot do?
T	transition model: where might an action lead?
R	reward: what immediate feedback is produced?
γ	discount factor: how strongly do future rewards matter?

Key idea

An MDP separates the world model from the decision objective, then connects them through a policy.

What Counts as a State?

A useful robot state might include:

- location or campus region;
- battery band;
- remaining delivery task;
- whether a door is open;
- congestion level if it changes future transitions.

The Markov property is

$$P(s_{t+1} \mid s_t, a_t, \text{history}) = P(s_{t+1} \mid s_t, a_t).$$

Symbols: P: probability; s_t, s_{t+1}: states at successive time steps; a_t: current action; $|$: given. History means earlier states and actions; the current state and action suffice to predict the next state.

Key idea

"State" is an engineering abstraction: include what matters for predicting the future, omit what does not.

State Design Is Already a Tradeoff

State too small

If state is only location, but battery changes which moves are safe, then two visits to the "same" state can have different futures.
The model is not truly Markov.

State too rich

Location $\times$ battery $\times$ task $\times$ crowd density $\times$ weather $\times$ door status $\times \cdots$
Prediction improves, but the number of states explodes.

Key idea

Markov state design trades model fidelity against state-space size.

Actions Are the Robot's Controllable Choices

The action set might include:

- move forward, turn, backtrack;
- wait for traffic to move;
- perform a slower scan or re-localize;
- deliver the package;
- dock or request assistance.

Key idea

The action set defines what control authority the agent actually has.

The Transition Model Makes Actions Stochastic

$$T(s, a, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$$

Symbols: s: current state; a: action; s': candidate next state (the prime means next, not a derivative); $T(s, a, s')$: its transition probability. t: time step; $P(\cdot \mid \cdot)$: conditional probability.

Example from the campus intersection:

Action	Possible next states
Shortcut	0.70 NearGoal, 0.30 CrowdedPlaza
Detour	0.95 SafeCorridor, 0.05 CrowdedPlaza
Wait	0.60 SafeCorridor, 0.40 Intersection

Key idea

The robot chooses the action, not the outcome.

Reward Is Immediate; Return Is Long-Term

Reward R_t

Feedback for one transition.

- $+20$ deliver package
- -1 each time step
- -8 enter dense crowd
- -100 collision

Return G_t

Accumulated reward produced by a sequence of future decisions.
A move can be mediocre now but create a much better long-term future.

Symbols: t: current time step; R_t: reward for one transition; G_t: accumulated future reward starting at time t. Return means accumulated reward, not a single immediate reward.

Key idea

MDPs optimize a trajectory of consequences, not the next reward in isolation.

Discounted Return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

Symbols: G_t: return from time t; R_{t+1}: next reward; R_{t+2}, R_{t+3}: later rewards; γ: discount factor; powers of γ reduce the weight of progressively later rewards; $\cdots$: continue the pattern.

- $\gamma = 0$: only the next reward matters.
- $0 < \gamma < 1$: future rewards matter but are progressively discounted.
- $\gamma \approx 1$: long-run consequences matter strongly.

Key idea

The discount factor controls how strongly delayed consequences influence current actions.

What Does the Discount Factor Mean for the Robot?

Time preference

Earlier task completion may be operationally more valuable.

Planning emphasis

A smaller γ emphasizes nearer rewards. Transition probabilities represent uncertain outcomes.

Mathematical control

For continuing tasks, $\gamma < 1$ keeps bounded discounted returns finite.

Key idea

γ is not "the probability the future exists." It is a modeling parameter controlling the value of delayed reward.

A Policy Is Not a Path

$$\pi(s) = a$$

Symbols: π: policy (decision rule); s: observed current state; a: action selected by the policy. $\pi(s) = a$ means choose action a in state s.

Path

Intersection → Plaza → Library → Goal.
One assumed state sequence.

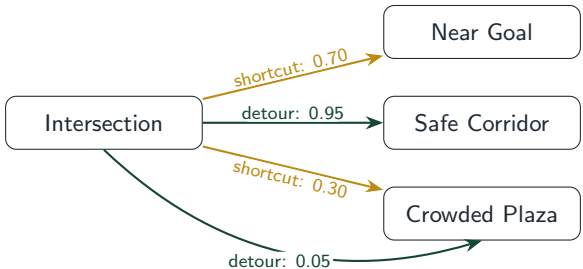
Policy

If crowd low → shortcut.
If crowd high → detour.
If battery low → dock.

Key idea

A policy specifies what to do from every state the robot might actually reach.

Possible Next States from the Intersection



Each reached state has its own later decisions and rewards. The figure shows the two movement actions; waiting and later transitions are omitted.

Key idea

Observe the state reached, then follow the policy from there. The probabilities for each action sum to one.

Immediate Reward Can Choose the Wrong Action

Shortcut:

$$R = +6$$

Detour:

$$R = +2$$

Symbols: R: immediate reward for the chosen action in this toy comparison. The values $+6$ and $+2$ do not include the rewards from later decisions.

A one-step agent prefers the shortcut immediately.
But if the shortcut often enters a terrible future state, the correct long-run decision can reverse.

Key idea

Scoring only the immediate reward omits the future decisions available from the next state.

Action Value Adds the Future

Given continuation values $V(s')$ for the next states:

$$Q(s,a) = \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V(s') \right]$$

Symbols: $Q(s,a)$: action value given continuation values V; s, a, s': current state, action, next state; T: transition probability; R: immediate transition reward; γ: discount factor; $V(s')$: continuation value; $\sum_{s'}$: sum over next states.

- T: how likely is each next state?
- R: what do we receive now?
- $V(s')$: how good is the future state we create?

Key idea

An action is good when it gives good reward now and tends to place the robot in valuable future states.

Bellman Optimality: The Core Recursion

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^*(s') \right]$$

Symbols: $V^*(s)$: optimal expected return from state s; $*$: optimal; a: candidate action; s': next state; T: transition probability; R: immediate reward; γ: discount; $\sum_{s'}$: sum over next states; $\max_a$: take the greatest action value.

Read it procedurally:

1. consider one candidate action;
2. consider every next state it can produce;
3. add immediate reward to discounted future value;
4. average by transition probability;
5. choose the action with the largest result.

Key idea

The value of a state is the value of making the best first move and then behaving optimally afterward.

Worked Comparison: Immediate Reward Plus Future Value

Use $\gamma = 0.9$ and these current continuation-value estimates:

Next state	Estimated future value
Near Goal	20
Safe Corridor	17
Crowded Plaza	-25

$$Q(\text{Intersection}, \text{Shortcut}) = 6 + 0.9[0.70(20) + 0.30(-25)] = 11.85$$

$$Q(\text{Intersection}, \text{Detour}) = 2 + 0.9[0.95(17) + 0.05(-25)] = 15.41$$

Symbols: $Q(s, a)$: immediate reward plus discounted expected continuation value; $\gamma = 0.9$: discount. $0.70, 0.30, 0.95, 0.05$: transition probabilities; $20, 17, -25$: values from the table; $6, 2$: immediate rewards for shortcut and detour, independent of next state.

Key idea

With these continuation values, detour wins despite its smaller immediate reward.

What Is a Value Function?

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t \mid S_t = s\right]$$

Symbols: $V^{\pi}(s)$: expected return when following policy π from state s; $\mathbb{E}_{\pi}$: average over trajectories generated by that policy; G_t: discounted return; $S_t = s$: start in state s at time t; V^*: optimal value.

$V^{\pi}(s)$ asks:

> *If I start in state s and follow policy π from now on, how much discounted reward should I expect?*

$V^*(s)$ asks the same question assuming optimal behavior from now on.

Key idea

Values compress many possible future trajectories into one reusable number.

Value Iteration: Repeated Bellman Backups

$$V_{k+1}(s) = \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

Symbols: k: update sweep, not a time step; V_k, V_{k+1}: old and new value estimates; s, a, s': state, action, next state; T: transition probability; R: immediate reward; γ: discount; $\sum_{s'}$: sum over next states; $\max_a$: best action value.

1. initialize values, often $V_0(s) = 0$;
2. perform a Bellman backup for each state;
3. repeat until values stabilize sufficiently.

Key idea

Value iteration converts a long-horizon planning problem into repeated local updates.

Intuition: Reward Propagates Backward



Symbols: V_0: initial value estimate before any update sweep; V: future return from a state; $+20$: reward on entering the goal. The terminal goal has $V = 0$ because no rewards remain after arrival.

In this separate, deterministic chain, all other rewards are zero and delivery pays $+20$ on entering the terminal goal.

- First sweep: Near Goal learns the value of delivering.
- Later sweeps carry discounted value back through Corridor and Intersection.

Key idea

The terminal state has no future reward. Dynamic programming reuses downstream values rather than enumerating every complete route.

Extract the Policy from the Values

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s') \right]$$

Symbols: $\pi^*(s)$: optimal action in state s; $V^*(s')$: optimal next-state value; T: transition probability; R: immediate reward; γ: discount. $\sum_{s'}$: sum over next states; $\arg\max_a$: return a maximizing action, not its numerical value.

Example policy:

- Intersection $\rightarrow$ Detour
- SafeCorridor $\rightarrow$ Forward
- CrowdedPlaza $\rightarrow$ Wait / Exit
- NearGoal $\rightarrow$ Deliver

Key idea

The value function scores states; the policy chooses the action that best exploits those values.

Policy Iteration: Another Route to the Same Goal

Policy evaluation

Hold a policy fixed and estimate how valuable every state is if the robot keeps following it.

Repeat until the policy no longer changes.

Policy improvement

At each state, check whether a different action would produce greater expected long-term value.

Key idea

Value iteration updates values directly; policy iteration alternates evaluating a policy and improving it.

What an MDP Quietly Assumes

Standard MDP planning assumes:

- the current modeled state is known;
- a transition model is available;
- the state/action space is manageable enough to represent values or policies.

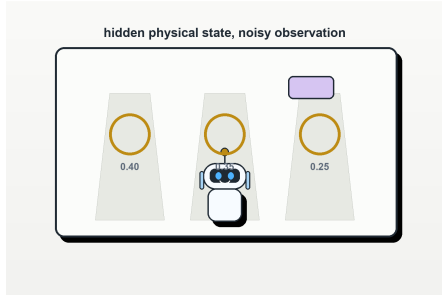
Key idea

MDPs solve sequential uncertainty in outcomes—but standard MDPs still assume the current state is observable.

Part 2: What If the Current State Is Hidden?

So far we have assumed the current modeled state is observed. In practice:

- localization can drift;
- two hallways may look alike;
- a camera may miss an occluded pedestrian;
- a door may be hidden around a corner.



Key idea

Uncertain action outcomes and an uncertain current state are different issues. A POMDP models both.

Full Observability Was Doing More Work Than It Looked Like

An MDP policy is

$$\pi(s).$$

But the robot may only know

$$P(CorridorA) = 0.55, \qquad P(CorridorB) = 0.45.$$

Symbols: $\pi(s)$: action prescribed for a known state s; $P(\text{CorridorA}), P(\text{CorridorB})$: current probabilities of being in those two locations. Neither location is known with certainty.

Which state should we plug into the MDP policy?

Key idea

If the current state is uncertain, a policy over physical states cannot be applied directly.

But Lecture 21 Already Gave Us the Missing Object

$$b_t(s) = P(S_t = s \mid o_{1:t}, a_{1:t-1})$$

The **belief state** is a probability distribution over possible physical states.
Example:

$$b_t = [0.55,\ 0.35,\ 0.10]$$

for Corridor A, Corridor B, and Plaza.

Symbols: $b_t(s)$: belief probability for state s at time t; S_t: hidden state variable; $o_{1:t}$: observations through t; $a_{1:t-1}$: earlier actions; $P(\cdot \mid \cdot)$: probability given that history. Vector entries follow the listed state order.

Key idea

State estimation and sequential decision making meet at the belief state.

Partially Observable Markov Decision Process

$$\boxed{POMDP = (S, A, T, R, \Omega, O, \gamma)}$$

Symbols: S: states; A: actions; T: transition model; R: reward model; γ: discount; Ω: observation set; $O(o \mid s', a)$: probability of reading o after action a reaches next state s'.

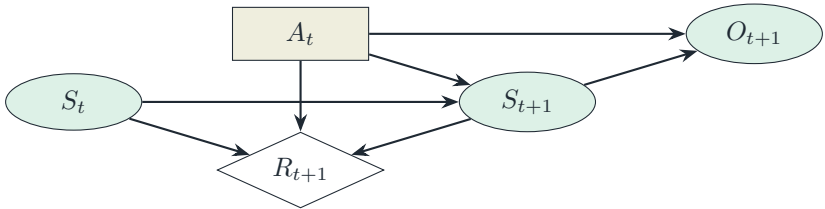
Relative to an MDP we add:

- Ω: the possible observations;
- $O(o \mid s', a)$: the observation model.

Key idea

A POMDP is an MDP where the agent acts without directly observing the underlying state.

The POMDP as a Dynamic Bayes Network



Symbols: S_t, S_{t+1}: hidden states; A_t: action; O_{t+1}: observation variable; R_{t+1}: reward; t: time step. The function $O(o \mid s', a)$ denotes the observation likelihood.

States are hidden; sensor readings are observed. Reward scores transitions; in this model we do not use a separate reward observation as evidence.

Key idea

Actions influence future states and sensor evidence. Planning must anticipate both, using the transition and observation models.

The Belief State Becomes the Planning State

Physical state:

where the robot actually is, whether the route is truly blocked, whether a door is truly open.

Belief state:

$$b = [0.55,\ 0.35,\ 0.10].$$

The controller now uses

$$\pi(b)$$

instead of $\pi(s)$.

Symbols: b: belief vector over Corridor A, Corridor B, and Plaza, in that order; $\pi(b)$: action chosen from the belief; $\pi(s)$: action chosen from a known physical state s.

Key idea

The robot plans from a probability distribution rather than pretending one hidden state is certainly true.

Belief Update: Predict, Then Correct

Prediction after action a_t:

$$b_{t+1}^{-}(s') = \sum_{s} T(s, a_t, s') b_t(s).$$

Correction after observation o_{t+1}:

$$b_{t+1}(s') \propto O(o_{t+1} \mid s', a_t) b_{t+1}^{-}(s').$$

Symbols: b_t: current belief; b_{t+1}^{-}: predicted belief before sensing; b_{t+1}: corrected belief; s, s': current and next states; a_t: action; o_{t+1}: new reading; T: transition model; O: observation likelihood; $\sum_s$: sum over current states; $\propto$: proportional to; normalize over next states to sum to one.

Key idea

Use the transition model to predict, then the observation likelihood to correct. This is filtering inside a sequential decision model.

The Elegant Trick: A POMDP Becomes an MDP over Beliefs



Symbols: b_t: current belief distribution; a_t: chosen action; o_{t+1}: new observation; b_{t+1}: updated belief after acting and observing; t: time step.

The full action/observation history can be compressed into the current belief.

Key idea

The belief is a sufficient statistic for POMDP control—but that elegance creates a new computational problem.

Problem #1: The Belief Space Is Continuous

Suppose there are only two possible hidden states: A and B.
A belief is

$$b = [P(A),\ 1 - P(A)].$$

Every value

$$0 \leq P(A) \leq 1$$

defines a different belief state.

Symbols: b: belief vector; $P(A)$: probability of hidden state A; $1 - P(A)$: probability of hidden state B. Here A and B label states, not action sets; $\leq$: less than or equal to.

Key idea

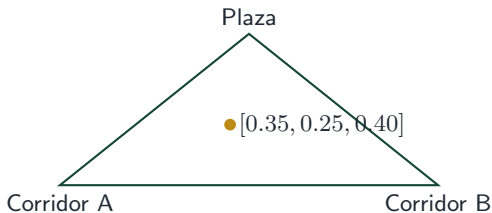
Two physical states already create infinitely many possible beliefs.

With Three Hidden States, Beliefs Fill a Triangle

For three hidden states,

$$b = [p_1, p_2, p_3], \qquad p_1 + p_2 + p_3 = 1.$$

Symbols: b: belief vector; p_1, p_2, p_3: nonnegative probabilities of Corridor A, Corridor B, and Plaza; their sum is 1. n: number of hidden states; $n - 1$: independent probability coordinates.



Key idea

For n hidden states, beliefs live in a continuous $(n - 1)$-dimensional probability simplex.

A POMDP Policy Maps Beliefs to Actions

Different belief regions can prefer different actions:

- highly confident in Corridor A $\rightarrow$ turn left;
- location ambiguous but uncertainty reducible $\rightarrow$ scan;
- likely near the plaza with high congestion $\rightarrow$ detour.

Key idea

The best action can change because the belief changes—even when the most likely physical state stays the same.

Problem #2: Future Observations Create Branching Contingencies

POMDP planning must reason about plans like:

Choose action a_1. If observation o_1 arrives, do a_2. If observation o_2 arrives instead, do a_3.

So the future branches on:

- actions;
- stochastic physical transitions;
- stochastic observations;
- resulting belief updates.

Key idea

POMDP planning is contingency planning over both uncertain world evolution and uncertain future information.

How Fast Does That Branching Grow?

A rough action–observation branching count is

$$(|A||\Omega|)^H.$$

With four actions, three coarse observation categories, and horizon $H = 6$:

$$(4 \cdot 3)^6 = 12^6 \approx 2.99 \times 10^6.$$

Symbols: $|A|$: number of actions; $|\Omega|$: number of observation categories; vertical bars mean set size; H: planning horizon in steps. $(|A||\Omega|)^H$ counts possible length-H action–observation sequences; $\approx$: approximately.

And every branch carries a continuously valued belief distribution.

Key idea

Horizon, action branching, observation branching, and continuous beliefs compound each other.

So We End Up Back at a Familiar Question

Do we really need to enumerate every possible future?

Exact approach

Represent and evaluate every relevant branch of the future.

Monte Carlo approach

Sample plausible futures, observe their returns, and estimate which actions look best.

Key idea

The same tradeoff returns: give up exact enumeration to gain tractable approximate decision making.

Monte Carlo Rollouts for Decision Making

A simulation-based online planner can:

1. sample a physical state from the current belief;
2. try a candidate action;
3. sample the next state and observation;
4. continue with a rollout policy that uses only the simulated observations or beliefs;
5. accumulate discounted reward;
6. repeat many times and average the return.

Key idea

Average sampled returns to estimate action value. The simulated controller must not use the hidden state as if it were observed.

The Pieces We Already Learned Can Be Reused

Particle belief

Represent the current belief with particles instead of an exact table.

Monte Carlo futures

Sample transitions and observations to generate possible trajectories.

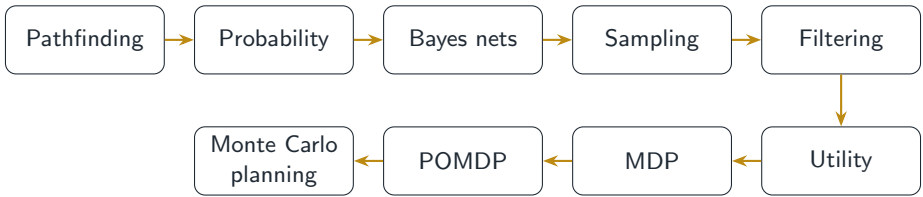
Tree search

Spend more simulation on promising choices rather than exploring everything uniformly.

Key idea

Approximate POMDP planning can combine sampled beliefs with Monte Carlo lookahead instead of constructing the entire belief-space solution.

The Unit Has Come Full Circle



We reuse earlier ideas as the agent handles more uncertainty and a longer decision horizon.

Key idea

Beliefs describe possible worlds; rewards score consequences; planning selects actions over time. Sampling makes large planning problems more tractable.

What Each Model Adds

Model	Decision input	Problem we solved
Lecture 22 examples	Current beliefs and utilities	Compare one route choice, with optional information before choosing.
MDP	Observed current state	Choose actions for expected accumulated reward over time.
POMDP	Belief over hidden states	Choose actions for accumulated reward, anticipating future observations and belief updates.

In class

A robot is unsure which hallway it is in and can pay to scan before continuing. Explain why a POMDP policy can favor scanning even though it earns no immediate delivery reward.